Practical 1: Simple Reflex Agent - Vacuum Cleaner World

Code

```
# Simple Reflex Agent - Vacuum Cleaner World
environment = {
    "A": "Dirty",
    "B": "Dirty"
}
location = "A"
steps = 6

print("Initial Environment:", environment)
print("Initial Vacuum Location:", location)
print("\nAgent Execution:\n")

for step in range(1, steps + 1):
    status = environment[location]
    print("Step", step)
    print("Location:", location)
    print("Room Status:", status)

    if status == "Dirty":
        action = "Suck"
        environment[location] = "Clean"

    elif location == "A":
        action = "Move Right"
        location = "B"

    else:
        action = "Move Left"
        location = "A"

    print("Action:", action)
    print("Environment:", environment)
    print("-" * 30)

print("\nFinal Environment:", environment)
print("Final Vacuum Location:", location)
```

Output

```
Initial Environment: {'A': 'Dirty', 'B': 'Dirty'}
Initial Vacuum Location: A

Agent Execution:

Step 1
Location: A
Room Status: Dirty
Action: Suck
Environment: {'A': 'Clean', 'B': 'Dirty'}
------------------------------
Step 2
Location: A
Room Status: Clean
Action: Move Right
Environment: {'A': 'Clean', 'B': 'Dirty'}
------------------------------
Step 3
Location: B
Room Status: Dirty
Action: Suck
Environment: {'A': 'Clean', 'B': 'Clean'}
------------------------------
Step 4
Location: B
Room Status: Clean
Action: Move Left
Environment: {'A': 'Clean', 'B': 'Clean'}
------------------------------
Step 5
Location: A
```

```
Room Status: Clean
Action: Move Right
Environment: {'A': 'Clean', 'B': 'Clean'}
------------------------------
Step 6
Location: B
Room Status: Clean
Action: Move Left
Environment: {'A': 'Clean', 'B': 'Clean'}
------------------------------

Final Environment: {'A': 'Clean', 'B': 'Clean'}
Final Vacuum Location: A
```

Practical 2: Tower of Hanoi - State Space Search (BFS)

Code

```
from collections import deque

def get_successors(state):
    successors = []
    for source in range(3):
        if len(state[source]) == 0:
            continue
        disk = state[source][-1]
        for destination in range(3):
            if source == destination:
                continue
            if (len(state[destination]) == 0 or
                    state[destination][-1] > disk):
                new_state = [list(rod) for rod in state]
                new_state[source].pop()
                new_state[destination].append(disk)
                new_state = tuple(tuple(rod) for rod in new_state)
                successors.append((new_state, source, destination, disk))
    return successors

def display_state(state):
    print("Rod A:", list(state[0]))
    print("Rod B:", list(state[1]))
    print("Rod C:", list(state[2]))
    print()

def tower_of_hanoi(n):
    initial_state = (tuple(range(n, 0, -1)), (), ())
    goal_state = ((), (), tuple(range(n, 0, -1)))

    queue = deque()
    queue.append((initial_state, []))
    visited = set()
    visited.add(initial_state)

    while queue:
        current_state, path = queue.popleft()

        if current_state == goal_state:
            return path + [(current_state, "Goal")]

        for successor, source, destination, disk in get_successors(current_state):
            if successor not in visited:
                visited.add(successor)
                move = (
                    f"Move disk {disk} from "
                    f"Rod {'ABC'[source]} to Rod {'ABC'[destination]}"
                )
                new_path = path + [(current_state, move)]
                queue.append((successor, new_path))
    return None

n = 3
solution = tower_of_hanoi(n)

if solution:
    print("Tower of Hanoi - State Space Search")
    print("Number of disks:", n)
    print("\nSolution Path:\n")

    initial_state = solution[0][0]
    print("Initial State:")
    display_state(initial_state)

    for i in range(len(solution) - 1):
        state, move = solution[i]
        print("Step", i + 1, ":", move)
        next_state = solution[i + 1][0]
        display_state(next_state)
```

```
    print("Goal State Reached!")
else:
    print("No solution found.")
```

Output

```
Tower of Hanoi - State Space Search
Number of disks: 3

Solution Path:

Initial State:
Rod A: [3, 2, 1]
Rod B: []
Rod C: []

Step 1 : Move disk 1 from Rod A to Rod C
Rod A: [3, 2]
Rod B: []
Rod C: [1]

Step 2 : Move disk 2 from Rod A to Rod B
Rod A: [3]
Rod B: [2]
Rod C: [1]

Step 3 : Move disk 1 from Rod C to Rod B
Rod A: [3]
Rod B: [2, 1]
Rod C: []

Step 4 : Move disk 3 from Rod A to Rod C
Rod A: []
Rod B: [2, 1]
Rod C: [3]

Step 5 : Move disk 1 from Rod B to Rod A
Rod A: [1]
Rod B: [2]
Rod C: [3]

Step 6 : Move disk 2 from Rod B to Rod C
Rod A: [1]
Rod B: []
Rod C: [3, 2]

Step 7 : Move disk 1 from Rod A to Rod C
Rod A: []
Rod B: []
Rod C: [3, 2, 1]

Goal State Reached!
```

Practical 3: Robot Path Planning using BFS

Code

```
from collections import deque

grid = [
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 0, 0, 0],
    [0, 0, 0, 0, 0]
]

start = (0, 0)
goal = (4, 4)

directions = [
    (-1, 0),   # Up
    (1, 0),    # Down
    (0, -1),   # Left
    (0, 1)     # Right
]

def bfs(grid, start, goal):
    queue = deque()
    queue.append(start)
    visited = set()
    visited.add(start)
    parent = {}
    parent[start] = None
    rows = len(grid)
    cols = len(grid[0])

    while queue:
        current = queue.popleft()

        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]
            path.reverse()
            return path

        for dr, dc in directions:
            new_row = current[0] + dr
            new_col = current[1] + dc

            if (0 <= new_row < rows and
                    0 <= new_col < cols and
                    grid[new_row][new_col] == 0):
                neighbor = (new_row, new_col)
                if neighbor not in visited:
                    visited.add(neighbor)
                    parent[neighbor] = current
                    queue.append(neighbor)
    return None

path = bfs(grid, start, goal)

if path:
    print("Robot Path Planning using BFS")
    print("\nGrid:")
    for row in grid:
        print(row)
    print("\nStart Position:", start)
    print("Goal Position:", goal)
    print("\nShortest Path:")
    for step, position in enumerate(path):
        print("Step", step, ":", position)
    print("\nPath Length:", len(path) - 1)
else:
    print("No path found.")
```

Output

```
Robot Path Planning using BFS

Grid:
[0, 0, 0, 0, 0]
[0, 1, 1, 1, 0]
[0, 0, 0, 1, 0]
[0, 1, 0, 0, 0]
[0, 0, 0, 0, 0]

Start Position: (0, 0)
Goal Position: (4, 4)

Shortest Path:
Step 0 : (0, 0)
Step 1 : (1, 0)
Step 2 : (2, 0)
Step 3 : (3, 0)
Step 4 : (4, 0)
Step 5 : (4, 1)
Step 6 : (4, 2)
Step 7 : (4, 3)
Step 8 : (4, 4)

Path Length: 8
```

Practical 4: Water Jug Problem using DFS

Code

```
# Water Jug Problem using DFS
CAPACITY_A = 4
CAPACITY_B = 3
GOAL = 2

def get_next_states(state):
    a, b = state
    states = []
    states.append(((CAPACITY_A, b), "Fill Jug A"))
    states.append(((a, CAPACITY_B), "Fill Jug B"))
    states.append(((0, b), "Empty Jug A"))
    states.append(((a, 0), "Empty Jug B"))

    amount = min(a, CAPACITY_B - b)
    states.append(((a - amount, b + amount), "Pour A -> B"))

    amount = min(b, CAPACITY_A - a)
    states.append(((a + amount, b - amount), "Pour B -> A"))
    return states

def dfs():
    initial = (0, 0)
    stack = [initial]
    visited = set()
    visited.add(initial)
    parent = {initial: None}
    action = {initial: "Initial State"}

    while stack:
        current = stack.pop()

        if current[0] == GOAL or current[1] == GOAL:
            path = []
            while current is not None:
                path.append((current, action[current]))
                current = parent[current]
            path.reverse()
            return path

        next_states = get_next_states(current)
        for next_state, move in next_states:
            if next_state not in visited:
                visited.add(next_state)
                parent[next_state] = current
                action[next_state] = move
                stack.append(next_state)
    return None

solution = dfs()

if solution:
    print("Water Jug Problem using DFS")
    print("Jug A Capacity =", CAPACITY_A, "liters")
    print("Jug B Capacity =", CAPACITY_B, "liters")
    print("Goal =", GOAL, "liters")
    print("\nSolution Path:\n")

    for step, (state, move) in enumerate(solution):
        print("Step", step)
        print("Action:", move)
        print("Jug A:", state[0], "liters")
        print("Jug B:", state[1], "liters")
        print("-" * 30)
else:
    print("No solution found.")
```

Output

```
Water Jug Problem using DFS
Jug A Capacity = 4 liters
```

```
Jug B Capacity = 3 liters
Goal = 2 liters

Solution Path:

Step 0
Action: Initial State
Jug A: 0 liters
Jug B: 0 liters
------------------------------
Step 1
Action: Fill Jug B
Jug A: 0 liters
Jug B: 3 liters
------------------------------
Step 2
Action: Pour B -> A
Jug A: 3 liters
Jug B: 0 liters
------------------------------
Step 3
Action: Fill Jug B
Jug A: 3 liters
Jug B: 3 liters
------------------------------
Step 4
Action: Pour B -> A
Jug A: 4 liters
Jug B: 2 liters
------------------------------
```

Practical 5: Rule-Based Chatbot using Pattern Matching

Code

```
# Rule-Based Chatbot using Pattern Matching
patterns = {
    "hello": "Hello! How can I help you?",
    "hi": "Hi! Nice to meet you.",
    "hey": "Hey! How are you?",
    "how are you": "I am fine. Thank you for asking!",
    "what is your name": "I am a simple rule-based chatbot.",
    "who are you": "I am a chatbot created using Python.",
    "help": "I can respond to simple greetings and questions.",
    "thank you": "You're welcome!",
    "thanks": "You're welcome!",
    "bye": "Goodbye! Have a nice day."
}

exit_commands = ["bye", "exit", "quit"]

print("==================================")
print("      Simple Rule-Based Chatbot")
print("==================================")
print("Chatbot: Hello! I am a simple chatbot.")
print("Chatbot: Type 'bye', 'exit' or 'quit' to end the conversation.\n")

while True:
    user_input = input("You: ")
    user_input = user_input.lower().strip()

    if user_input in exit_commands:
        print("Chatbot: Goodbye! Have a nice day.")
        break

    matched = False
    for pattern, response in patterns.items():
        if pattern in user_input:
            print("Chatbot:", response)
            matched = True
            break

    if matched == False:
        print("Chatbot: Sorry, I don't understand that.")
```

Output

```
==================================
      Simple Rule-Based Chatbot
==================================
Chatbot: Hello! I am a simple chatbot.
Chatbot: Type 'bye', 'exit' or 'quit' to end the conversation.

You: hello
Chatbot: Hello! How can I help you?
You: what is your name
Chatbot: I am a simple rule-based chatbot.
You: how are you?
Chatbot: I am fine. Thank you for asking!
You: thank you
Chatbot: You're welcome!
You: what is python?
Chatbot: Sorry, I don't understand that.
You: bye
Chatbot: Goodbye! Have a nice day.
```